1.

```
def vacuum_cleaner(loc, status):
    if status[loc] == "Dirty":
        print(f"Room {loc} is dirty. Action --> Suck")
        status[loc] = "Clean"
    else:
        print(f"Room {loc} is clean.")
        loc = "B" if loc == "A" else "A"
        print(f"Action --> Moving to Room {loc}")
    return loc, status


loc = input("Start location (A/B): ").upper()
status = {
    "A": input("Status of Room A: ").capitalize(),
    "B": input("Status of Room B: ").capitalize()
}

print("\nInitial status:", status)
print("Initial location:", loc, "\n")

while status["A"] == "Dirty" or status["B"] == "Dirty":
    loc, status = vacuum_cleaner(loc, status)

print("\nBoth rooms are clean. Action --> No Operation")
```

2.

```
grid = [
   ['s', 'X', '.', '.', '.'],
   ['.', 'X', '.', 'X', '.'],
   ['.', 'X', '.', 'X', '.'],
   ['.', 'X', '.', 'X', '.'],
   ['.', '.', '.', 'X', 'F']
]

row=0
column=0

visited_nodes=[]

while True:
   visited_nodes.append((row,column))
   print("Current Postion : ",(row,column))

   if grid[row][column]=="F":
      print("Goal Reached!!")
      break

   if column < 4 and grid[row][column+1] != 'X' and (row, column+1) not in visited_nodes:
      column += 1
   elif row < 4 and grid[row+1][column] != 'X' and (row+1, column) not in visited_nodes:
      row += 1
   elif row > 0 and grid[row-1][column] != 'X' and (row-1, column) not in visited_nodes:
      row -= 1
   elif column > 0 and grid[row][column-1] != 'X' and (row, column-1) not in visited_nodes:
      column -= 1
   else:
      print("No Path Found")
      break

print("Visited Cells :", visited_nodes)
```

3.

```
import heapq

goal = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))

def heuristic(state):
    h = 0
    for i in range(3):
        for j in range(3):
            num = state[i][j]
            if num != 0:
                x = (num - 1) // 3
                y = (num - 1) % 3
                h += abs(i - x) + abs(j - y)
    return h

def neighbors(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j
    moves = []
    directions = [(-1,0), (1,0), (0,-1), (0,1)]
    for dx, dy in directions:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            temp = [list(row) for row in state]
            temp[x][y], temp[nx][ny] = temp[nx][ny], temp[x][y]
            moves.append(tuple(tuple(row) for row in temp))
    return moves

def astar(start):
    pq = []
    heapq.heappush(pq, (heuristic(start), 0, start, []))
    visited = set()
    while pq:
        f, g, state, path = heapq.heappop(pq)
        if state in visited:
            continue
        visited.add(state)
        path = path + [state]

        if state == goal:
```

```
            return path
        for nxt in neighbors(state):
            if nxt not in visited:
                heapq.heappush(pq, (g + 1 + heuristic(nxt), g + 1, nxt, path))
    return None

def display(state):
    for row in state:
        print(row)
    print()

start = ((1, 2, 3),
         (4, 7, 5),
         (6, 8, 8))

result = astar(start)
if result:
    print("Solution found in", len(result) - 1, "moves\n")
    for i, state in enumerate(result):
        print("Step", i)
        display(state)
else:
    print("No solution found.")
```

4.

```
from collections import deque

def get_neighbors(state):
    moves = []
    for src in range(3):
        for dst in range(3):
            if src != dst and state[src]:
                disk = state[src][0]
                if not state[dst] or state[dst][0] > disk:
                    new_state = [list(rod) for rod in state]
                    new_state[dst].insert(0, new_state[src].pop(0))
                    moves.append(tuple(tuple(rod) for rod in new_state))
    return moves

def solve_hanoi(n):
    start = (tuple(range(1, n + 1)), (), ())
    goal = ((), (), tuple(range(1, n + 1)))

    queue = deque([(start, [])])
    visited = {start}

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        for nxt in get_neighbors(state):
            if nxt not in visited:
                visited.add(nxt)
                queue.append((nxt, path + [state]))

    return None

def display(state):
    for i, rod in enumerate(state):
        print(f"Rod {i + 1}: {rod}")
    print()

num_disks = 3
result = solve_hanoi(num_disks)

if result:
    print("Solution found in", len(result) - 1, "moves\n")
```

```
    for i, state in enumerate(result):
        print("Step", i)
        display(state)
else:
    print("No solution found.")
```

5.

```
from collections import deque

def get_successor(state):
    successor = []
    stacks = [list(stack) for stack in state]

    for i in range(len(stacks)):
        if not stacks[i]:
            continue

        block = stacks[i][-1]

        for j in range(len(stacks)):
            if i == j:
                continue

            new_stacks = [s[:] for s in stacks]
            new_stacks[i].pop()
            new_stacks[j].append(block)

            result_state = tuple(tuple(s) for s in new_stacks if s)
            if result_state not in successor:
                successor.append(result_state)

        new_stacks = [s[:] for s in stacks]
        new_stacks[i].pop()
        new_stacks.append([block])

        result_state = tuple(tuple(s) for s in new_stacks if s)
        if result_state not in successor:
            successor.append(result_state)

    return successor

def bfs(start, goal):
    queue = deque()
    queue.append((start, []))

    visited = set()
    visited.add(start)

    while queue:
        current, path = queue.popleft()
```

```
        if current == goal:
            return path + [current]

        for next_state in get_successor(current):
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [current]))

    return None

def print_state(state):
    print("-" * 25)
    for i, stack in enumerate(state):
        print(f"Stack {i + 1} : {list(stack)}")
    print("-" * 25)

start = (('C', 'B', 'D', 'A'),)
goal = (('A', 'B', 'C', 'D'),)

solution = bfs(start, goal)

if solution:
    print("\nSolution Found!")
    for step, state in enumerate(solution):
        print(f"Step {step}")
        print_state(state)
else:
    print("No Solution Found")
```

6.

```
def is_safe(board, row, col):
    for i in range(row):
        if board[i] == col or abs(board[i] - col) == abs(i - row):
            return False
    return True

def solve_n_queens(n, row, board, solutions):
    if row == n:
        solutions.append(board[:])
        return True

    for col in range(n):
        if is_safe(board, row, col):
            board[row] = col
            if solve_n_queens(n, row + 1, board, solutions):
                return True
    return False

def print_board(board):
    n = len(board)
    for row in range(n):
        line = ["Q" if col == board[row] else "." for col in range(n)]
        print(" ".join(line))
    print("-" * 15)

n = 8
solutions = []
board = [-1] * n

solve_n_queens(n, 0, board, solutions)

if solutions:
    print("Solution:")
    print_board(solutions[0])
else:
    print("No solution found.")
```

7.

```
print("========================================")
print(" Welcome to AI Rule-Based Chatbot")
print("Type 'bye' to end the conversation.")
print("========================================")
while True:
    user_input = input("\nYou : ").lower().strip()
    if user_input == "hello" or user_input == "hi":
        print("Bot : Hello! How can I help you?")
    elif user_input == "how are you":
        print("Bot : I am fine. Thank you!")
    elif user_input == "what is your name":
        print("Bot : My name is AI Chatbot.")
    elif user_input == "who developed you":
        print("Bot : I was developed for learning Artificial Intelligence.")
    elif user_input == "what is ai":
        print("Bot : AI stands for Artificial Intelligence.")
    elif user_input == "help":
        print("Bot : I can answer simple predefined questions.")
    elif user_input == "thanks" or user_input == "thank you":
        print("Bot : You're welcome!")
    elif user_input == "bye":
        print("Bot : Goodbye! Have a nice day.")
        break
    else:
        print("Bot : Sorry, I don't understand your question.")
```

8.

```
board = [' '] * 9
WIN_LINES = [(0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8), (0,4,8), (2,4,6)]

def print_board():
    print("\n" + "\n---+---+---\n".join(
        f" {board[i]} | {board[i+1]} | {board[i+2]} " for i in range(0, 9, 3)
    ) + "\n")

def get_winner():
    for a, b, c in WIN_LINES:
        if board[a] == board[b] == board[c] != ' ':
            return board[a]
    return None

def minimax(is_ai_turn):
    winner = get_winner()
    if winner == 'O': return 1, None
    if winner == 'X': return -1, None
    if ' ' not in board: return 0, None

    best_score = -float('inf') if is_ai_turn else float('inf')
    best_move = None

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O' if is_ai_turn else 'X'
            score, _ = minimax(not is_ai_turn)
            board[i] = ' '

            if is_ai_turn and score > best_score:
                best_score, best_move = score, i
            elif not is_ai_turn and score < best_score:
                best_score, best_move = score, i

    return best_score, best_move

print("Positions: 1-9 (left-to-right, top-to-bottom)")

while ' ' in board and not get_winner():
    print_board()

    while True:
        try:
```

```
            pos = int(input("Enter position (1-9): ")) - 1
            if board[pos] == ' ':
                board[pos] = 'X'
                break
        except (ValueError, IndexError):
            pass
        print("Invalid move. Try again.")

    if get_winner() or ' ' not in board:
        break

    _, ai_pos = minimax(True)
    if ai_pos is not None:
        board[ai_pos] = 'O'

print_board()
winner = get_winner()
print(f"{winner} Wins!" if winner else "It's a Draw!")
```